

SQL PRACTICE QUESTIONS

1. departments Table

dept_id department_name

1	Engineering
2	Human Resources
3	Finance
4	Marketing
5	Data Science

2. employees Table

emp_id name dept_id salary hire_date

101	Ravi Kumar	1	75000	2022-01-15
102	Neha Sharma	5	68000	2023-05-10
103	Kiran Reddy	3	52000	2021-11-25
104	Sneha Patel	2	48000	2023-03-19
105	Arjun Singh	1	92000	2020-09-03
106	Vishwa Rao	5	85000	2024-02-12
107	Samuel Peter	4	56000	2022-12-01

3. customers Table

customer_id customer_name city email

1	Anand	Hyderabad	anand@example.com
2	Priya	Mumbai	priya123@example.com
3	Ramesh	Chennai	ramesh@example.com
4	Kavya	Delhi	kavya@example.com

customer_id	customer_name	city	email
5	John	Bangalore	john@gmail.com

4. orders Table

order_id	customer_id	order_date	amount
201	1	2024-01-10	1200
202	3	2024-02-14	3500
203	1	2024-02-25	2100
204	4	2024-03-05	1500
205	2	2024-03-15	800
206	5	2024-03-30	4500
207	3	2024-04-01	900

5. products Table

product_id	product_name	price
501	Laptop	55000
502	Mobile	20000
503	Headphones	2500
504	Keyboard	1500
505	Monitor	12000

6. order_items Table (Bridge Table)

(Links orders → products)

item_id	order_id	product_id	quantity
1	201	503	2

item_id	order_id	product_id	quantity
2	202	501	1
3	203	502	1
4	204	504	3
5	205	503	1
6	206	501	1
7	207	505	1

1 Get all employees with their department names.

```
SELECT e.name, d.department_name
```

```
FROM employees e
```

```
JOIN departments d ON e.dept_id = d.id;
```

2 Find the second-highest salary.

```
SELECT MAX(salary)
```

```
FROM employees
```

```
WHERE salary < (SELECT MAX(salary) FROM employees);
```

3 Count employees in each department.

```
SELECT dept_id, COUNT(*)
```

```
FROM employees
```

```
GROUP BY dept_id;
```

4 .Get employees earning above average salary.

```
SELECT *
```

```
FROM employees
```

```
WHERE salary > (SELECT AVG(salary) FROM employees);
```

5 Show top 5 highest salaries.

```
SELECT name, salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 5;
```

6 Find duplicate emails.

```
SELECT email, COUNT(*)  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

7 Get employees who do NOT belong to any department.

```
SELECT *  
FROM employees  
LEFT JOIN departments ON employees.dept_id = departments.id  
WHERE departments.id IS NULL;
```

8 Get total salary per department.

```
SELECT dept_id, SUM(salary)  
FROM employees  
GROUP BY dept_id;
```

9 Find customers who placed more than 3 orders.

```
SELECT customer_id, COUNT(*)  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(*) > 3;
```

10. Get employees hired in the last 30 days.

```
SELECT *  
  
FROM employees  
  
WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days';
```

PART 2 — PostgreSQL-Specific Practice (WITH ANSWERS)

PostgreSQL has UNIQUE features like arrays, JSON, ILIKE, DISTINCT ON, UPSERT, window functions.

1 Case-insensitive search (PostgreSQL only)

```
SELECT * FROM users  
  
WHERE name ILIKE '%vishwa%';
```

2 Select first row per department (DISTINCT ON)

```
SELECT DISTINCT ON (dept_id) *  
  
FROM employees  
  
ORDER BY dept_id, salary DESC;
```

3 Update or Insert (UPSERT)

```
INSERT INTO users(id, name)  
  
VALUES (1, 'Vishwa')  
  
ON CONFLICT (id)  
  
DO UPDATE SET name = EXCLUDED.name;
```

4 Extract year from date

```
SELECT EXTRACT(YEAR FROM hire_date)  
  
FROM employees;
```

5 Use JSON field

```
SELECT data->>'email' AS email  
FROM customers;
```

6 Unnest array values

```
SELECT unnest(skills)  
FROM employees;
```

7 Ranking employees by salary (Window function)

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC)  
FROM employees;
```

8 Count rows per year

```
SELECT EXTRACT(YEAR FROM order_date) AS year,  
       COUNT(*)  
FROM orders  
GROUP BY year;
```

9 Convert string to timestamp

```
SELECT to_timestamp('2024-01-01 10:00', 'YYYY-MM-DD HH24:MI');
```

10. Check table structure

```
\d employees
```

PART 3 — SQL Interview Questions (FAST MEMORIZATION SET)

These are asked in **ALL AI/ML, Data Science, and Data Engineer** interviews.

1. What is the difference between WHERE and HAVING?

- WHERE filters rows before GROUP BY
 - HAVING filters groups after GROUP BY
-

2. Difference between INNER JOIN and LEFT JOIN?

- INNER JOIN → only matching rows
 - LEFT JOIN → all left rows + matching from right
-

3. What is a primary key?

A column that uniquely identifies each row (no duplicates, no NULL).

4. What is a foreign key?

A column linking one table to another.

5. What is normalization?

Organizing data to reduce duplication & improve integrity.

6. What is denormalization?

Adding redundancy to improve query speed.

7. What does GROUP BY do?

Combines rows with the same value into groups.

8. Purpose of ORDER BY?

Sort the results (ASC/DESC).

9. What is an index?

A performance optimization for faster search.

10. What is a subquery?

A query inside another query.

11. What is UNION vs UNION ALL?

- UNION removes duplicates
 - UNION ALL keeps duplicates
-

12. What is DISTINCT?

Removes duplicate rows.

13. What is ACID?

Atomicity, Consistency, Isolation, Durability.

14. What is a view?

A virtual table based on a query.

15. What is a stored procedure?

Pre-defined SQL code that can be executed repeatedly.

16. What is a trigger?

SQL code that runs automatically on INSERT / UPDATE / DELETE.

17. What is DELETE vs TRUNCATE?

- DELETE = slow, logs each row
 - TRUNCATE = fast, removes all rows
-

18. What is LIMIT?

Restricts number of returned rows.

19. What is COALESCE?

Returns the first non-null value.

20. What is a window function?

Allows calculations over a group of rows **without** collapsing them (RANK, LAG, LEAD).

PART 4 — PostgreSQL Interview Questions

1. What is the difference between LIKE and ILIKE?

- LIKE is case-sensitive
 - ILIKE is case-insensitive
-

2. What is DISTINCT ON in PostgreSQL?

Fetch only 1 unique row per key.

3. What is JSONB in PostgreSQL?

Binary JSON format — searchable, indexable, faster.

4. What is an extension?

Plugins like pgcrypto, postgis, etc.

5. How to check table schema?

`\d table_name`

6. What is UPSERT?

Insert + Update combined.

7. Does PostgreSQL support arrays?

Yes.

Example column type: `TEXT[]`

8. What is CTE (WITH clause)?

Temporary result set used in a query.

9. How to create a sequence?

`CREATE SEQUENCE my_seq;`

10. What is VACUUM?

Cleans up dead tuples, improves performance.